EPFL - Computer Science

CS-401 Applied Data Analysis

Lecture 07 — October 31, 2025

Author: Jonathan Dutoit

Professor: Maria Brbic

Learning from data: Supervised learning

Machine Learning

- Supervised: We are given input/output pairs (X, y) that are related via a function y = f(X). We would like to "learn" f, and evaluate it on new data. The types are discrete "classification" and continuous "regression"
- Unsupervised: Given only samples X of the data, we computer a function f such that y = f(X) is a "simpler" representation. The types are discrete "clustering" and continuous "dimensionality reduction"

Learning goals

1. k-NN (k nearest neighbors)
2. Tree-based models: decision trees, random forests
3. Logistic regression

K nearest neighbors (k-NN)

Given a sample, we'd like to find the k closest matches in a labelled dataset. The we return the most frequent label among the k

The data is the model
- No training needed.
- Conceptually simple algorithm.
- Accuracy generally improves with more data.
- Usually need data in memory, but can also be run from disk.

Minimal configuration:
- Only one parameter: k (number of neighbors)
- But two other choices are also important
  - Similarity metric
  - Weighting of neighbors in voting (e.g., by similarity)

Classification
- Model is y = f(X), y is from a discrete set (labels)
- Given X, compute y = majority vote of the k nearest neighbors
- Can also use a weighted vote of the neighbors

Regression
- Model is y = f(X), y is a real value
- Given X, compute y = average value of the k nearest neighbors.
- Can also use a weighted average of the neighbors.

k-NN distance measures

- Euclidean distance: simplest, fast to compute
$$d(x, y) = ||x - y||$$
- Cosine Distance: good for documents, images, etc.
$$d(x, y) = 1 - \frac{x \cdot y}{||x|| \cdot ||y||}$$
- Jaccard Distance: for set data:
$$d(X, Y) = 1 - \frac{|X \cap Y|}{|X \cup Y|}$$
- Hamming Distance: for string data:
$$d(x, y) = \sum_{i=1}^{n} (x_i \neq y_i)$$
- Manhattan Distance: coordinate-wise distance
$$d(x, y) = \sum_{i=1}^{N} |x_i - y_i|$$
- Edit Distance: for strings, especially genetic data.

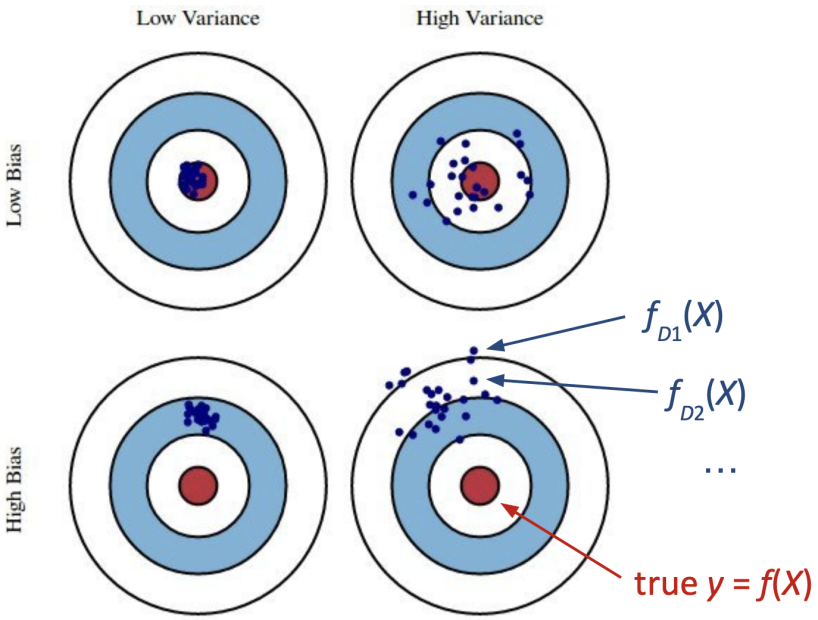
Predicting from samples

Most datasets are samples from a (maybe infinite) population. We are most interest in models of the population, but we only have access to a samples from the population.

Hence, we train on a training sample D and denote the fitted models as f_D. For a new data point (X, y), predictions if $f_D(X)$

Our data-generated model $f_D(X)$ is a statistical estimate of the true function $f(X)$. Because of this, its subject to bias and variance:

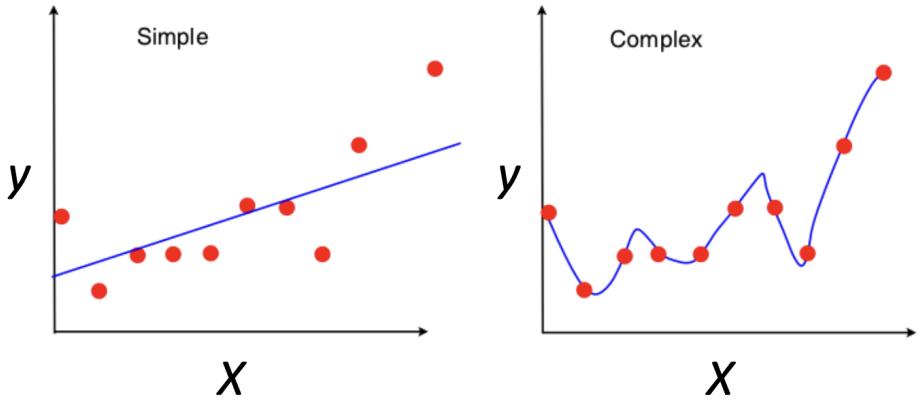
- (Squared) error = $\mathrm{E}[(f_D(X) - y)^2]$ (E is expectation over D)
- Fact: error can be decomposed into two parts
  - $Error^2 = Bias^2 + Variance$
  - Bias = $\mathrm{E}[f_D(X)$ - y] that represents how far off we are on average from the true value. We can overpredict or underpredict.
  - Variance = $E[(f_D(X)$ - $\mathrm{E}[f_D(X)])^2]$ that represents the variance of the estimates.



Bias/variance tradeoff

Since $Error^2 = Bias^2 + Variance$, there is a tradeoff, usually modulated via model complexity. Hence complex models (many parameters) usually have lower bias, but higher variance. Simple models (few parameters) have higher bias, but lover variance.

Example: a linear model can only fit a straight line. A high-degree polynomial can fit a complex curve. But the polynomial will fit the individual training sample, rather than the full population. Its shape can vary from sample to sample, so it has high variance.



- if *variance* strongly dominates, it means there is too much variation between models. This is called over-fitting
- if *bias* strongly dominates, then the models are not fitting the data well enough. This is called under-fitting

Choosing k for k nearest neighbors

- Lower $k \Rightarrow$ high variance, low bias (if the k is equals to 1, it will always output the predicted value, so no neighbors into account)
- Higher $k \Rightarrow$ low variance, high bias (if k is the equal to the sample size, then it outputs always the average)

Choosing k in practice

Use leave-one-out (LOO) cross-validation

- Split: Break data into train and test subsets, e.g., 80-20 % random split.
- Predict: For each point in the training set, predict using the k nearest neighbors from the set of all *other* points in training set. Measure the LOO error rate (classification) or squared error (regression).
- Tune: Try different values of k, and use the one that gives minimum leave-one-out error.
- Evaluate: Measure error on the test set to quantify performance.

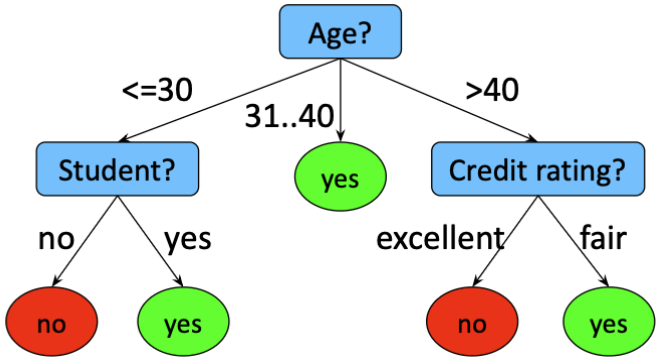
Decision trees

Decision tree learning is a supervised learning approach used in statistics, data mining and machine learning.

In this formalism, a classification or regression decision tree is used as a predictive model to draw conclusions about a set of observations.

Tree models where the target variable can take a discrete set of values are called classification trees; in these tree structures, leaves represent class labels and branches represent conjunctions of features that lead to those class labels.

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31..40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31..40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31..40	medium	no	excellent	yes
31..40	high	yes	fair	yes
>40	medium	no	excellent	no



Goal: Find decision tree that maximizes classification accuracy on given dataset

Optimization: NP-hard! Heuristic: greedy top-down tree construction + pruning

Formal Definition

Given a dataset $D = \{(x_i, y_i)\}_{i=1}^{N}$

- At each node, the algorithm selects a **feature j** and a **threshold t** that best separates the classes (or minimizes prediction error in regression)

We define impurity (for classification) using metrics like:

- Entropy

$$H(D) = -\sum_{k=1}^{K} p_k \log_2(p_k).$$

For regression:

$$MSE(D, j, t) = \frac{1}{|D_L|} \sum_{i \in D_L} (y_i - \hat{y_L})^2 + \frac{1}{|D_R|} \sum_{i \in D_R} (y_i - \hat{y_R})^2.$$

and we choose the split minimizing MSE.

Tree Growth and Stopping

- **Recursive Binary Splitting**: Split nodes greedily until a stopping condition:
  - Max depth
  - Min samples per leaf
  - Min impurity decrease
- **Pruning**: To prevent overfitting, prune the tree using:

- Cost Complexity Pruning (CART): minimize

$$C_\alpha(T) = R(T) + \alpha|T|.$$

Where $R(T)$ is misclassification cost and $|T|$ is number of leaves.

Advantages and Disadvantages

Pros:

- Easy to interpret and visualize
- Handles numerical and categorical features
- Requires little data preprocessing (no scaling, normalization)

Cons:

- High variance (overfits easily)
- Small changes in data can change the tree structure drastically
- Greddy nature means not globally optimal.

Ensemble methods

Are, metaphorically, like "democratic" machine learning algorithms:

- Take a collection of simple or *weak* learners
- Combine their results to make a single, better learner

Types:

- Bagging: train learners in parallel on different samples of the data, then combine by voting (for discrete output) or by averaging (for continuous output)
- Boosting: train learner again, but after filtering/weighting samples based on output of previous train/test runs
- Stacking: combine outputs from various models using a second-stage learner (e.g., linear regression).

Random forests

Grow K trees on datasets sampled from the original dataset (size N) with replacement (bootstrap samples), p = number of features.

Principles: we want to take a vote between different learners so we don't want the models to be too similar. The following two criteria (blue comments) ensure diversity in the individual trees:

- Draw K bootstrap samples of size N.
  - Each tree is trained on different data.
- Grow each decision tree by selecting a random set of m out of p features at each node and choosing the best feature to split on.
  - Corresponding nodes in different trees (usually) can't use the same feature to split on.
- At testing time, aggregate the predictions of the trees (most popular vote, or average) to produce the final class (example of bagging).

Typically m might be e.g., `sqrt(p)`, but can be smaller.

- Very popular in practice, probably the most popular classifier for dense data (up to a few thousand features)

- Easy to implement (simply train many normal decision trees)
- Easy to parallelize
- Needs many passes over the data - at least the max depth of the trees

Boosted decision trees

A more recent alternative to random forests. In contrast to RF, whose trees are trained independently by bagging, BDT trees are trained **sequentially** by **boosting**: Each tree is trained to predict ("correct") residual errors of previous trees ($\Rightarrow$ bias reduction).

Final prediction: sum of predictions made by individual trees.

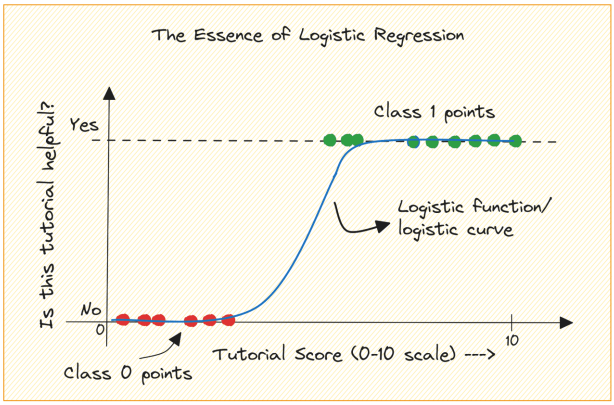
- Both RF and BDT can produce very high-quality models. Superiority of one method or the other is dataset-dependent.
- Resource requirements are very different as well, so it's actually non-trivial to compare the methods.
- The "geometry" of the methods is very different. For instance, RF use 10's of deep, large trees. Bias reductions comes through depth and variance reduction through the ensemble aggregate. Boosted decision trees use 1000's of shallow, small trees. Bias reduction through boosting, variance is already low.

Logistic regression

In statistic, a **logistic model** is a statistical model that models the **log-odds** of an event as a **linear combination** of one or more independent variables. Hence, in regression analysis, logistic regression estimates the parameters of a logistic model.

It's a supervised machine learning algorithm widely used for **binary classification tasks**. This approach utilizes the logistic (or sigmoid) function to transform linear combination of input features into a probability value ranging between 0 and 1.

This probability indicates the **likelihood** that a given input corresponds to one of two predefined categories.



With its distinctive S-shaped curve, the logistic function effectively maps any real-valued number to a value within the 0 to 1 interval. This feature renders it particularly suitable for binary classification tasks. The logistic (sigmoid) function is of the form:

$$p(x) = \frac{1}{1 + e^{-(x-\mu)/s}}.$$

where μ is a location parameter (the midpoint of the curve, where $p(\mu) = 1/2$ and s is a scale parameter. This expression may be rewritten as:

$$p(x) = \frac{1}{1 + e^{-(\beta_0+\beta_1 x)}}.$$

where $\beta_0 = -\mu/s$ and is known as the intercept (it is the vertical intercept or y-intercept of the line $y = \beta_0 + \beta_1 x$), and $\beta_1 = 1/s$ (inverse scale parameter or rate parameter): these are the y-intercept and slope of the log-odds as a function of x.

Optimization: finding best model via maximum likelihood.

- Don't use square loss as in linear regression (where y is assumed to be generated from Normal distribution)
- Use cross-entropy loss instead (y assumed to be generated from Bernoulli distribution, i.e., biased coin)

The more features the better? No!. More features mean less bias, but more variance, which implies overhitting.

Carefully selected features can improve model accuracy.

- E.g, keep features that correlate with the label y
- Forward/backward feature selection
- Regularization (e.g., penalize norm of weight vector)